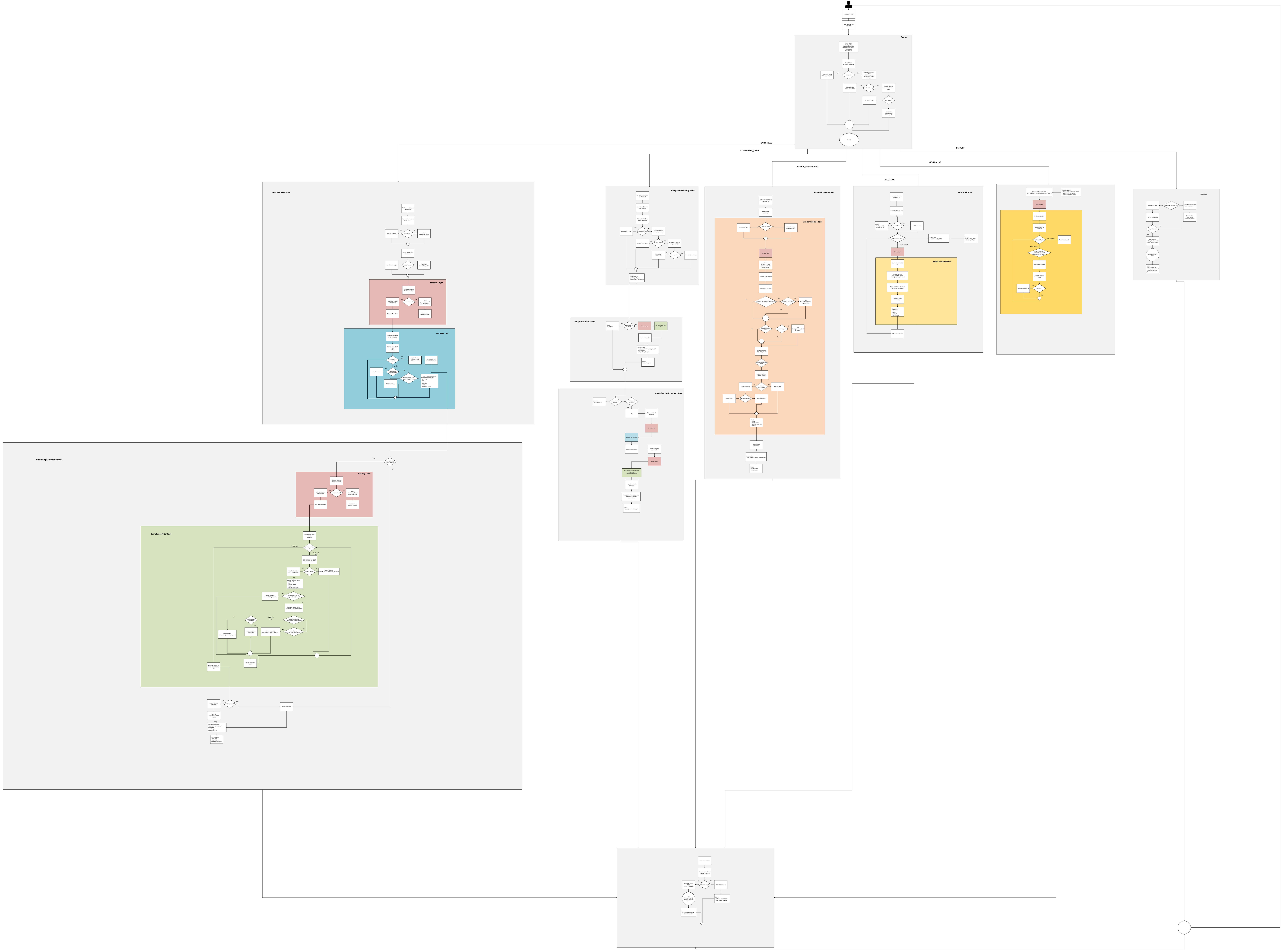
User Query or Input
Fetch User Type and Session ID
Router
Define intent:
SALES_RECO,
COMPLIANCE_CHECK,
VENDOR_ONBOARDING,
OPS_STOCK,
GENERAL_KB
Score intents
by keyword matching
score > 0 ?
True
False
Return Best Intent
routed_by = keyword
Regex-Based Memory Check:
Detect follow-up actions using basket/ cart regex
Basket Follow-up ?
Yes
No
Return DEFAULT
routed_by=memory
Call LLM to decide Router based on User Input
LLM Success ?
Return DEFAULT
Return LLM Decision intent
routed_by = llm
Output
SALES_RECO
COMPLIANCE_CHECK
VENDOR_ONBOARDING
OPS_STOCK
GENERAL_KB
DEFAULT
Sales Hot Picks Node
Get Session Information by session_id
Extract State from User Input _state_u
State Found ?
Yes
No
Use Extracted State
Use Session Memory last_state
Extract Budget from User Input
Budget Found ?
Use Extracted Budget
Use Session Memory last_budget
Security Layer
Tool Call Permission Check by user_type
Access Allowed ?
Yes
No
Audit: tool_invoked
Log tool usage
Audit: permission_denied
Log denied access
Start Tool Timer/Trace
Raise Exception PermissionDenied
Hot Picks Tool
Load Product Catalog rows = products[]
Initialize Empty Result List
out = []
Loop Through Each Product
End Loop
A Product
Sort Products by Popularity Score
Highest -> Lowest
Apply Result Limit
return top N products
Budget Filter
price > budget ?
Yes
No
Skip The Product
State Restriction Filter
state in blocked_states ?
Skip The Product
Add Product to Output (out)
Append Product Metadata
- product_id
- sku
- name
- category
- price
- popularity_score
Sales Compliance Filter Node
State Exists AND Product IDs Exist ?
Yes
No
Security Layer
Tool Call Permission Check by user_type
Access Allowed ?
Yes
No
Audit: tool_invoked
Log tool usage
Audit: permission_denied
Log denied access
Start Tool Timer/Trace
Raise Exception PermissionDenied
Compliance Filter Tool
Initialize Empty Results List,
results = []
Loop Through Product IDs
End of Loop
A Product ID (pid)
Fetch Product from Catalog
prod = product_by_id(pid)
Product Found ?
Yes
No
Normalize State Code
state_u = state.upper()
Append to Result:
REVIEW, reason=UNKNOWN_PRODUCT
Extract Product Metadata
- product_id
- sku
- blocked_states
- flags
- lab_report_required
Check Blocked States List
state_u in blocked_states ?
Yes
No
Return BLOCKED
reason=STATE_BLOCKED
Load State Restricted Flags
from STATE_FLAG_RESTRICTIONS
Loop on Product Flags
THC / Kratom / Mushroom / etc.
End of the Loop
If Product Flag
in STATE_FLAG_RESTRICTIONS?
Yes
No
Check Lab Report Required ?
Yes
No
Return REVIEW,
reason = LAB_REPORT_REQUIRED
Return ALLOWED,
reason=OK
Return BLOCKED,
Reason = STATE_FLAG_RESTRICTED
Append Decision to Results[]
Return Legality Results
ALLOWED / BLOCKED / etc.
Legality has Results?
Yes
No
Extract ALLOWED Product IDs
Filter Picks
Keep only ALLOWED products
Use Original Picks
Persist Session Memory
- last_intent = SALES_RECO
- last_state
- last_budget
- last_product_ids
Return Response
- final_picks
- legality report
- filtered product_ids
Compliance Identify Node
Get Session Information by session_id
Extract State from User Input : state_u
Extract product IDs by SKU in User Input
Are product IDs found?
Yes
No
matched_by = "sku"
Match products by name in User Input
Products found by name?
Yes
No
matched_by = "name"
Load session memory:
last_product_ids
Products in memory?
Yes
No
matched_by = "memory"
matched_by = "none"
Return:
"state_code": st,
"product_ids": pids,
"matched_by": matched_by
Compliance Filter Node
Missing state or products?
Yes
No
Return:
"legality": []
Security Layer
Call Compliance Filter Tool
Get legality results
Persist session:
last_intent = "COMPLIANCE_CHECK"
last_state = st
last_product_ids = pids
Return:
"legality": legality
Compliance Alternatives Node
Missing state or legality?
Are all products ALLOWED?
No
Return:
"alternatives": []
No
Get session data by session_id
Security Layer
Call Sales Hot Picks Tool
Get candidate products
Extract candidate product IDs
Security Layer
Re-check legality of candidate products by Compliance Filter Tool
Keep only ALLOWED product IDs
Filter candidates by allowed IDs
alternatives = allowed candidates[:3]
Return:
"alternatives": alternatives
Vendor Validate Node
Get Session Information by session_id
Extract vendor attributes
Vendor Validate Tool
Attributes found?
Use extracted attrs
Use fallback attrs
state.vendor_attrs
Security Layer
Check
REQUIRED_FIELDS
missing = fields with missing values
Initialize required_docs = []
Get category from attrs
Category in LAB_REPORT_CATEGORIES ?
Yes
No
lab_report_url missing ?
Yes
No
Add
"LAB_REPORT_COA" to required_docs
"state_restrictions" exists ?
Yes
Is it a list type ?
Yes
No
Add
"state_restrictions[list]" to missing
Build checklist for REQUIRED_FIELDS
Category requires lab report?
Add lab_report_url status to checklist
missing OR required_docs ?
yes
No
Find hard_missing
status = "PASS"
hard_missing exists ?
status="FAIL"
status="REVIEW"
Return:
- status
- missing_fields
- required_documents
- checklist
Store result in vendor_result
Persist session
- last_intent = VENDOR_ONBOARDING
Return:
- vendor_attrs,
- vendor_result
Ops Stock Node
Get Session Information by session_id
Extract Product IDs, Pids
Product IDs Found?
Yes
No
Return:
"stock_rows": []
"product_ids": []
Initialize rows = []
Loop for each Product ID
A Product ID
Persist session
- last_intent = OPS_STOCK
Return:
- "stock_rows": rows
- "product_ids": pids
Security Layer
Stock by Warehouse
Fetch product details by pid
Initialize rows list
Filter inventory records
where r["product_id"] == pid
Create warehouse-qty objects
{"warehouse": ..., "qty": ...}
Sort rows by qty descending
Return Result:
- product_id
- sku
- name
- total_qty
- warehouses
Add result to rows list
Get user visibility permission
vis = _VISIBILITY_BY_USER.get(state["user_type"])
Visibility Mapping:
internal_sales → None (all access)
portal_vendor → "vendor"
portal_customer → "public"
Security Layer
Tokenize User Query
Initialize scored list
scored = []
Loop through kb_docs
End of Loop
A Document
Return top_k results
Check visibility filter
visibility != None AND
doc["visibility"] != visibility ?
Tokenize document text
Calculate similarity score
score >= 0 ?
Yes
No
Add result to scored list
Default Node
Load session data
Check basket follow-up intent
Create fallback response
"I couldn't classify..."
Return output
+ tool_results
{"note":"no_intent"}
Get last_product_ids
last_pids exists ?
Yes
No
Build payload
basket_added = first PID
include session context
Generate assistant reply
Return:
- output = LLM text
- tool_results = payload
- product_ids = first PID
Get intent from state
Generate payload using _payload_for(state)
"error" in payload?
No
Yes
Get system prompt from _FORMAT_SYSTEMS
Map error message
Call format_with_llm()
Generate formatted response
Return:
- output = error message
- tool_results = payload
Return:
- output = formatted text
- tool_results = payload